

1. 서론

1.1 실험 목적

정렬 알고리즘은 동일한 데이터를 정렬하더라도 알고리즘의 동작 방식에 따라 실행 시간과 메모리 사용량 등의 성능이 달라질 수 있다. 본 보고서에서는 삽입 정렬(Insertion Sort), 병합 정렬(Merge Sort), 힙 정렬(Heap Sort)을 대상으로 동작 원리와 성능을 비교한다.

각 알고리즘의 시간 복잡도와 공간 복잡도 등의 이론적 특성을 살펴본 후, 동일한 조건에서 실행하여 데이터 크기에 따른 실행 시간의 변화를 측정한다. 또한 안정성(Stable Sort), In-place 여부 및 입력 데이터의 특성에 따른 성능 차이를 비교하여 각 알고리즘의 특징을 분석한다.

1.2 비교 대상 선정 이유

본 보고서에서는 수업에서 학습한 삽입 정렬과 병합 정렬, 수업에서 다루지 않은 힙 정렬을 비교 대상으로 선정하였다.

삽입 정렬은 구현이 간단하지만 평균 및 최악의 경우 $O(n^2)$ 의 시간 복잡도를 가지며, 입력 데이터의 정렬 상태에 따라 성능 차이가 크다.

병합 정렬은 분할 정복(Divide and Conquer) 방식으로 동작하며 일반적으로 $O(n \log n)$ 의 시간 복잡도를 가진다.

힙 정렬은 힙(Heap) 자료구조를 이용하며 $O(n \log n)$ 의 시간 복잡도를 가진다. 또한 병합 정렬과 달리 입력 배열 내부에서 정렬할 수 있다는 특징이 있다.

따라서 세 알고리즘을 비교함으로써 $O(n^2)$ 과 $O(n \log n)$ 의 시간 복잡도 차이와 함께 공간 복잡도, 안정성 및 정렬 방식의 차이를 확인하고자 한다.

2. 비교 대상 알고리즘 구동 방식

2.1 사용한 초기 데이터

세 가지 알고리즘을 동일한 조건에서 비교하기 위해 다음과 같은 초기 데이터를 사용하였다.

[37, 12, 25, 37, 8, 19, 4, 25, 31, 16, 8, 42, 3, 19, 27]

총 15개의 원소로 구성되어 있으며, 37, 25, 8, 19와 같이 중복된 값을 포함한다. 세 알고리즘에는 동일한 초기 배열의 복사본을 입력하여 공정하게 비교하였다.

2.2 삽입 정렬

삽입 정렬(Insertion Sort)은 배열의 원소를 하나씩 확인하면서 이미 정렬된 부분의 적절한 위치에 현재 원소를 삽입하는 방식으로 동작한다.

첫 번째 원소를 정렬된 상태로 간주한 후, 다음 원소부터 앞쪽의 정렬된 원소와 비교한다. 현재 원소보다 큰 값은 뒤로 이동시키고, 현재 원소가 들어갈 위치에 삽입하는 과정을 반복한다.

삽입 정렬은 최선의 경우 $O(n)$, 평균 및 최악의 경우 $O(n^2)$ 의 시간 복잡도를 가진다. 또한 입력 배열 내부에서 정렬하므로 추가 공간 복잡도는 $O(1)$ 이다.

2.3 병합 정렬

병합 정렬(Merge Sort)은 분할 정복(Divide and Conquer) 방식의 정렬 알고리즘이다. 배열을 절반씩 나누어 하나의 원소가 될 때까지 분할한 후, 정렬된 부분 배열을 다시 병합하여 전체 배열을 정렬한다.

예를 들어 [37]과 [12]를 병합하면 [12, 37]이 된다. 이러한 과정을 반복하여 전체 배열을 정렬한다.

병합 정렬은 최선, 평균, 최악의 경우 모두 $O(n \log n)$ 의 시간 복잡도를 가지며, 병합 과정에서 추가 배열을 사용하기 때문에 일반적인 배열 기반 구현에서는 $O(n)$ 의 추가 공간이 필요하다.

2.4 힙 정렬

힙 정렬(Heap Sort)은 힙(Heap) 자료구조를 이용하여 정렬하는 알고리즘이다. 본 보고서에서는 최대 힙을 사용하여 정렬한다.

먼저 배열을 최대 힙으로 구성한 후, 가장 큰 원소인 루트와 배열의 마지막 원소를 교환한다. 이후 남은 원소에 대해 다시 힙 조건을 만족하도록 조정하는 과정을 반복하여 정렬한다.

힙 정렬은 최선, 평균, 최악의 경우 모두 일반적으로 $O(n \log n)$ 의 시간 복잡도를 가지며, 입력 배열 내부에서 정렬할 수 있어 추가 공간 복잡도는 $O(1)$ 이다.

3. 삽입, 병합, 힙정렬 비교

3.1 시간 복잡도 비교

위에서 언급했듯이, 세 정렬 알고리즘은 이론적으로 서로 다른 시간 복잡도를 가진다. 삽입 정렬은 평균 및 최악의 경우 $O(n^2)$ 의 시간 복잡도를 가지며, 병합 정렬과 힙 정렬은 $O(n \log n)$ 의 시간 복잡도를 가진다. 이러한 이론적 차이가 실제 실행시간에서 어떻게 나타나는지 확인하기 위해 세 알고리즘을 동일한 조건에서 실행하여 비교하였다.

실험에서는 **100개, 1,000개, 5,000개, 10,000개의 랜덤 정수 데이터**를 각각 생성하여 데이터 크기에 따른 실행시간의 변화를 측정하였다. 각 데이터 크기마다 하나의 원본 배열을 생성하고, 동일한 원본 데이터를 복사하여 세 알고리즘에 입력함으로써 알고리즘마다 동일한 조건에서 비교가 이루어지도록 하였다.

또한 각 알고리즘은 동일한 데이터에 대해 **5회씩 실행**하고 평균 실행시간을 계산하였다. 실행시간 측정에는 실제 정렬 과정에 소요되는 시간만 포함하고, 데이터 생성 및 배열 복사에 소요되는 시간은 제외하였다. 이를 통해 데이터의 크기가 증가함에 따라 각 알고리즘의 실행시간이 어떻게 변화하는지 관찰하고, 실제 실험 결과가 이론적인 시간 복잡도와 어떠한 관계를 보이는지 비교하고자 하였다

3.1.1 100개 데이터

너가 0~999까지의 범위에서 랜덤한 100개짜리 정수 배열을 하나 만들어줘. 2개 이상 숫자의 중복 사례가 무조건 3개 이상 발생하게끔 설계해서.

2개 이상의 동일한 숫자가 나타나는 중복 사례가 정확히 3개 이상 포함된 100개짜리 정수 배열입니다.

중복 사례 예시: 42 (2회), 128 (3회), 753 (2회)

python

```
[
  42, 128, 753, 512, 890, 14, 637, 209, 945, 381,
  42, 618, 274, 803, 128, 956, 421, 67, 539, 712,
  305, 849, 162, 753, 694, 88, 923, 451, 128, 576,
  234, 681, 915, 348, 702, 159, 826, 493, 75, 614,
  938, 267, 580, 143, 872, 329, 650, 901, 416, 82,
  735, 298, 561, 184, 827, 370, 643, 917, 482, 59,
  764, 215, 532, 198, 865, 341, 679, 904, 438, 91,
  720, 253, 589, 167, 814, 395, 628, 971, 402, 36,
  783, 226, 519, 173, 857, 304, 692, 936, 475, 21,
  749, 282, 540, 111, 838, 363, 607, 982, 469, 54
]
```

📄 ↕ 🔍 🗑️ 🔒 ...

내 조직의 채팅은 Gemini 모델을 개선하는 데 사용되지 않습니다. Gemini는 사람이 아닙니다. 인물 등에 관한 정보 제공 시 실수를 할 수 있으니 다시 한번 확인하세요. [개인 정보 보호 및 Gemini](#)

Gemeni 모델에게 0~999 사이의 랜덤한 100개의 미정렬된 수를 중복포함하여 선택하도록 하였다. 이 초기 데이터를 그대로 깃허브의 삽입,병렬,힙정렬에 투입하고 (mysort_0000sort_TimeComplexity.py), 정렬 전 배열, 정렬 후 배열, 1~5회차 실행 속도, 1~5회차 평균 실행 속도를 각각 출력토록 하였다.

```

● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_heapsort_TimeComplexity.py
정렬 전 원본 배열:
[42, 128, 753, 512, 890, 14, 637, 209, 945, 381, 42, 618, 274, 803, 128, 956, 421, 67, 539, 712, 305, 849, 162, 753, 694, 88, 923, 451, 128, 576, 234, 681, 915, 348, 702, 159, 826, 493, 75, 614, 938, 267, 580, 143, 872, 329, 650, 901, 416, 82, 735, 298, 561, 184, 827, 370, 643, 917, 482, 59, 764, 215, 532, 198, 865, 341, 679, 904, 438, 91, 720, 253, 589, 167, 814, 395, 628, 971, 402, 36, 783, 226, 519, 173, 857, 304, 692, 936, 475, 21, 749, 282, 540, 111, 838, 363, 607, 982, 469, 54]
1회차 정렬 시간: 0.156061 ms
2회차 정렬 시간: 0.125965 ms
3회차 정렬 시간: 0.122669 ms
4회차 정렬 시간: 0.149489 ms
5회차 정렬 시간: 0.126837 ms
정렬 후 결과 배열:
[14, 21, 36, 42, 42, 54, 59, 67, 75, 82, 88, 91, 111, 128, 128, 128, 143, 159, 162, 167, 173, 184, 198, 209, 215, 226, 234, 253, 267, 274, 282, 298, 304, 305, 329, 341, 348, 363, 370, 381, 395, 402, 416, 421, 438, 451, 469, 475, 482, 493, 512, 519, 532, 539, 540, 561, 576, 580, 589, 607, 614, 618, 628, 637, 643, 650, 679, 681, 692, 694, 702, 712, 720, 735, 749, 753, 753, 764, 783, 803, 814, 826, 827, 838, 849, 857, 865, 872, 890, 901, 904, 915, 917, 923, 936, 938, 945, 956, 971, 982]
오름차순 정렬 검증: PASS
5회 실행시간 평균: 0.136204 ms
○ root@7c96e0182597:/work/src#

```

->힙정렬

```

● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_insertionsort_TimeComplexity.py
정렬 전 원본 배열:
[305, 849, 162, 753, 694, 88, 923, 451, 128, 576, 234, 681, 915, 348, 702, 159, 826, 493, 75, 614, 938, 267, 580, 143, 872, 329, 650, 901, 416, 82, 735, 298, 561, 184, 827, 370, 643, 917, 482, 59, 764, 215, 532, 198, 865, 341, 679, 904, 438, 91, 720, 253, 589, 167, 814, 395, 628, 971, 402, 36, 783, 226, 519, 173, 857, 304, 692, 936, 475, 21, 749, 282, 540, 111, 838, 363, 607, 982, 469, 54]
1회차 정렬 시간: 0.149619 ms
2회차 정렬 시간: 0.145471 ms
3회차 정렬 시간: 0.146052 ms
4회차 정렬 시간: 0.141674 ms
5회차 정렬 시간: 0.140643 ms
정렬 후 결과 배열:
[21, 36, 54, 59, 75, 82, 88, 91, 111, 128, 143, 159, 162, 167, 173, 184, 198, 215, 226, 234, 253, 267, 282, 298, 304, 305, 329, 341, 348, 363, 370, 395, 402, 416, 438, 451, 469, 475, 482, 493, 519, 532, 540, 561, 576, 580, 589, 607, 614, 628, 643, 650, 679, 681, 692, 694, 702, 720, 735, 749, 753, 764, 783, 814, 826, 827, 838, 849, 857, 865, 872, 901, 904, 915, 917, 923, 936, 938, 971, 982]
오름차순 정렬 검증: PASS
5회 실행시간 평균: 0.144692 ms
○ root@7c96e0182597:/work/src#

```

->삽입정렬

```

● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_mergesort_TimeComplexity.py
정렬 전 원본 배열:
[305, 849, 162, 753, 694, 88, 923, 451, 128, 576, 234, 681, 915, 348, 702, 159, 826, 493, 75, 614, 938, 267, 580, 143, 872, 329, 650, 901, 416, 82, 735, 298, 561, 184, 827, 370, 643, 917, 482, 59, 764, 215, 532, 198, 865, 341, 679, 904, 438, 91, 720, 253, 589, 167, 814, 395, 628, 971, 402, 36, 783, 226, 519, 173, 857, 304, 692, 936, 475, 21, 749, 282, 540, 111, 838, 363, 607, 982, 469, 54]
1회차 정렬 시간: 0.108322 ms
2회차 정렬 시간: 0.095539 ms
3회차 정렬 시간: 0.088685 ms
4회차 정렬 시간: 0.096260 ms
5회차 정렬 시간: 0.148339 ms
정렬 후 결과 배열:
[21, 36, 54, 59, 75, 82, 88, 91, 111, 128, 143, 159, 162, 167, 173, 184, 198, 215, 226, 234, 253, 267, 282, 298, 304, 305, 329, 341, 348, 363, 370, 395, 402, 416, 438, 451, 469, 475, 482, 493, 519, 532, 540, 561, 576, 580, 589, 607, 614, 628, 643, 650, 679, 681, 692, 694, 702, 720, 735, 749, 753, 764, 783, 814, 826, 827, 838, 849, 857, 865, 872, 901, 904, 915, 917, 923, 936, 938, 971, 982]
오름차순 정렬 검증: PASS
5회 실행시간 평균: 0.107429 ms
○ root@7c96e0182597:/work/src#

```

->머지정렬

알고리즘	1회 (ms)	2회 (ms)	3회 (ms)	4회 (ms)	5회 (ms)	평균 (ms)	📄
삽입 정렬	0.149619	0.145471	0.146052	0.141674	0.140643	0.144692	
병합 정렬	0.108322	0.095539	0.088685	0.096260	0.148339	0.107429	
힙 정렬	0.156061	0.125965	0.122669	0.149489	0.126837	0.136204	

측정 결과, 삽입 정렬의 평균 실행시간은 0.144692 ms, 병합 정렬은 0.107429 ms, 힙 정렬은 0.136204 ms로 나타났다. 100개의 비교적 작은 데이터에서는 세 알고리즘의 실행시간 차이가 크지 않았으며, 병합 정렬이 가장 짧은 평균 실행시간을 보였다.

다만 100개의 데이터만으로는 각 알고리즘의 시간 복잡도에 따른 성능 차이가 크게 나

타나지 않는 모습이다.

3.1.2 1000개 데이터

랜덤한 1000개의 숫자부터는 Gemini 모델에게 요구하기에 한계가 있어 Github에 mycode_generaterandomintegerdata 파일을 만들어 랜덤한 숫자를 뽑아내게끔 만들었다. 그렇게 생성된 초기 데이터를 그대로 깃허브의 삽입,병렬,힙정렬에 투입하고 (mysort_0000sort_TimeComplexity.py), 정렬 전 배열, 정렬 후 배열, 1~5회차 실행 속도, 1~5회차 평균 실행 속도를 각각 출력토록 하였다.

```
● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_heapsort_TimeComplexity.py
1회차 정렬 시간: 1.788167 ms
2회차 정렬 시간: 1.777237 ms
3회차 정렬 시간: 1.702197 ms
4회차 정렬 시간: 1.776405 ms
5회차 정렬 시간: 2.403126 ms
오름차순 정렬 검증: PASS
5회 실행시간 평균: 1.889426 ms
○ root@7c96e0182597:/work/src#
```

→힙정렬

```
root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_insertionsort_TimeComplexity.py
1회차 정렬 시간: 20.642150 ms
2회차 정렬 시간: 23.098578 ms
3회차 정렬 시간: 26.314715 ms
4회차 정렬 시간: 23.789134 ms
5회차 정렬 시간: 22.073281 ms
오름차순 정렬 검증: PASS
5회 실행시간 평균: 23.183572 ms
root@7c96e0182597:/work/src#
```

→삽입정렬

```
● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_mergesort_TimeComplexity.py
1회차 정렬 시간: 1.758973 ms
2회차 정렬 시간: 3.664427 ms
3회차 정렬 시간: 2.029438 ms
4회차 정렬 시간: 2.496059 ms
5회차 정렬 시간: 2.462286 ms
오름차순 정렬 검증: PASS
5회 실행시간 평균: 2.482237 ms
○ root@7c96e0182597:/work/src#
```

→병합정렬

알고리즘	1회 (ms)	2회 (ms)	3회 (ms)	4회 (ms)	5회 (ms)	평균 (ms)
삽입 정렬	20.642150	23.098578	26.314715	23.789134	22.073281	23.183572
병합 정렬	1.758973	3.664427	2.029438	2.496059	2.462286	2.482237
힙 정렬	1.788167	1.777237	1.702197	1.776405	2.403126	1.889426

측정 결과, 삽입 정렬의 평균 실행시간은 **23.183572 ms**, 병합 정렬은 **2.482237 ms**, 힙 정렬은 **1.889426 ms**로 나타났다. 100개 데이터를 대상으로 한 실험과 비교하면 데이터 크기가 증가하면서 세 알고리즘의 실행시간이 모두 증가하였으며, 특히 삽입 정렬의 실행시간 증가가 두드러졌다.

1,000개 데이터에서는 삽입 정렬과 병합 정렬 및 힙 정렬 사이의 실행시간 차이가 100개 데이터에서보다 크게 나타났다. 이는 데이터의 크기가 증가할수록 각 알고리즘의 시간 복잡도에 따른 실행시간 차이가 점차 나타날 수 있음을 보여준다.

3.1.3 10000개 데이터

랜덤한 10000개의 숫자역시 Gemini 모델에게 요구하기에 한계가 있어 Github에 mycode_generaterandomintegerdata 파일을 만들어 랜덤한 숫자를 뽑아내게끔 만들었다. 그 후 생성된 10000개의 초기데이터 정렬을 '10000random_integer_data' 항목에 저장한 후 깃허브의 삽입,병렬,힙정렬에 투입하고(mysort_0000sort_TimeComplexity.py), 정렬 전 배열, 정렬 후 배열, 1~5회차 실행 속도, 1~5회차 평균 실행 속도를 각각 출력토록 하였다.

```

● root@7c96e0182597:/work# cd "/work/src/" && python3 -u mysort_heapsort_TimeComplexity.py
1회차 정렬 시간: 29.345886 ms
2회차 정렬 시간: 27.856927 ms
3회차 정렬 시간: 47.588414 ms
4회차 정렬 시간: 44.826732 ms
5회차 정렬 시간: 27.439441 ms
오름차순 정렬 검증: PASS
5회 실행시간 평균: 35.411480 ms
○ root@7c96e0182597:/work/src#

```

→힙정렬

```

● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_insertionsort_TimeComplexity.py
1회차 정렬 시간: 3425.875738 ms
2회차 정렬 시간: 2898.431287 ms
3회차 정렬 시간: 2578.375825 ms
4회차 정렬 시간: 2570.054316 ms
5회차 정렬 시간: 2471.330187 ms
오름차순 정렬 검증: PASS
5회 실행시간 평균: 2788.813471 ms
○ root@7c96e0182597:/work/src#

```

→삽입정렬

```
root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_mergesort_TimeComplexity.py
1회차 정렬 시간: 27.279980 ms
2회차 정렬 시간: 27.532101 ms
3회차 정렬 시간: 38.164016 ms
4회차 정렬 시간: 32.086712 ms
5회차 정렬 시간: 47.358498 ms
오름차순 정렬 검증: PASS
5회 실행시간 평균: 34.484261 ms
root@7c96e0182597:/work/src#
```

→병합정렬

측정 결과, 삽입 정렬의 평균 실행시간은 **2788.813471 ms**, 병합 정렬은 **34.484261 ms**, 힙 정렬은 **35.411480 ms**로 나타났다. 특히 삽입 정렬은 병합 정렬과 힙 정렬에 비해 실행시간이 크게 증가하였으며, 데이터 크기가 증가할수록 세 알고리즘 간의 실행시간 차이가 더욱 크게 나타났다.

이는 삽입 정렬의 평균 시간 복잡도인 $O(n^2)$ 과 병합 정렬 및 힙 정렬의 $O(n \log n)$ 의 차이가 큰 입력 데이터에서 실제 실행시간의 차이로 나타난 것으로 볼 수 있다.

3.1.4 최종 분석

데이터 크기	삽입 정렬 (ms)	병합 정렬 (ms)	힙 정렬 (ms)
100	0.144692	0.107429	0.136204
1,000	23.183572	2.482237	1.889426
10,000	2,788.813471	34.484261	35.411480

실험 결과, 데이터의 크기가 증가할수록 세 알고리즘의 실행시간이 모두 증가하였다. 그러나 증가 폭에서는 알고리즘별 차이가 뚜렷하게 나타났다. 특히 삽입 정렬은 데이터가 100개에서 10,000개로 증가하면서 평균 실행시간이 크게 증가하였으며, 10,000개 데이터에서는 약 2.79초가 소요되었다. 반면 병합 정렬과 힙 정렬은 10,000개 데이터에서도 약 34~35ms의 실행시간을 보였다.

따라서 이번 실험에서는 데이터 크기가 증가할수록 삽입 정렬과 $O(n \log n)$ 알고리즘 사이의 실행시간 차이가 크게 나타나는 것을 확인할 수 있었다. 이는 이론적으로 예상한 시간 복잡도의 차이가 실제 실행 결과에서도 관찰된 것으로 볼 수 있다.

3.2 공간 복잡도 비교

시간 복잡도와 함께 정렬 알고리즘이 정렬 과정에서 추가적으로 사용하는 메모리의 크기도 비교하였다. 공간 복잡도는 입력 데이터 자체를 저장하는 데 필요한 공간을 제외하고, 정렬 과정에서 추가적으로 요구되는 메모리를 기준으로 비교하였다. 세 알고리즘의 공간 복잡도는 구현 방식에 따라 차이가 있으며, 특히 병합 정렬은 배열을 병합하는 과정에서 추가적인 메모리를 필요로 한다는 특징이 있다.

알고리즘	추가 공간 복잡도	주요 특징
삽입 정렬	$O(1)$	별도의 큰 배열 없이 원본 배열 내부에서 정렬
병합 정렬	$O(n)$	병합 과정에서 임시 배열 필요
힙 정렬	$O(1)$	원본 배열 내부에서 힙을 구성하여 정렬

3.3 안정성 비교

정렬 알고리즘의 안정성은 정렬 기준이 동일한 원소들의 상대적인 순서가 정렬 후에도 유지되는지를 의미한다. 본 실험에서는 동일한 점수를 가진 원소를 구분하기 위해 각 데이터에 식별자를 부여하고, 점수만을 기준으로 정렬한 후 동일한 점수를 가진 원소들의 순서가 유지되는지 확인하였다.

실험 데이터는 (80, A), (70, B), (80, C), (60, D), (80, E)를 사용하였다. 이 중 80점인 원소들의 정렬 전 순서는 $A \rightarrow C \rightarrow E$ 이며, 정렬 후에도 이 순서가 유지되는지를 확인하였다. 각각 'mysort_0000sort.stable.py'에 초기데이터를 집어넣었다.

```
● root@7c96e0182597:/work# cd "/work/src/" && python3 -u mysort_heap_stable.py
정렬 전 데이터: (80, A), (70, B), (80, C), (60, D), (80, E)
정렬 후 데이터: (60, D), (70, B), (80, C), (80, E), (80, A)
동일한 점수인 80점 원소들의 정렬 전 순서: A, C, E
정렬 후 80점 원소들의 순서: C, E, A
Stable: FAIL
○ root@7c96e0182597:/work/src#
```

```
● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_insertion_stable.py
정렬 전 데이터: (80, A), (70, B), (80, C), (60, D), (80, E)
정렬 후 데이터: (60, D), (70, B), (80, A), (80, C), (80, E)
동일한 점수인 80점 원소들의 정렬 전 순서: A, C, E
정렬 후 80점 원소들의 순서: A, C, E
Stable: PASS
○ root@7c96e0182597:/work/src# []
```

```

● root@7c96e0182597:/work/src# cd "/work/src/" && python3 -u mysort_merge_stable.py
정렬 전 데이터: (80, A), (70, B), (80, C), (60, D), (80, E)
정렬 후 데이터: (60, D), (70, B), (80, A), (80, C), (80, E)
동일한 점수인 80점 원소들의 정렬 전 순서: A, C, E
정렬 후 80점 원소들의 순서: A, C, E
Stable: PASS
○ root@7c96e0182597:/work/src#

```

실험 결과, 삽입 정렬과 병합 정렬은 80점 원소의 기존 순서인 $A \rightarrow C \rightarrow E$ 를 그대로 유지하여 안정적인 정렬임을 확인하였다. 반면 힙 정렬은 정렬 후 80점 원소의 순서가 $C \rightarrow E \rightarrow A$ 로 변경되어 기존의 상대적인 순서를 유지하지 못하였다.

따라서 본 실험에서는 삽입 정렬과 병합 정렬은 Stable Sort이고, 힙 정렬은 Stable Sort가 아님을 확인하였다. 힙 정렬의 경우 힙을 구성하고 원소를 교환하는 과정에서 동일한 값을 가진 원소들의 상대적인 순서가 변경될 수 있기 때문에 안정성이 보장되지 않는다.

3.4 In-place 정렬 여부 비교

In-place 정렬이란 입력 배열 외에 추가적인 큰 메모리 공간을 사용하지 않고 배열 내부에서 정렬을 수행하는 방식을 의미한다.

삽입 정렬은 입력 배열 내부에서 원소를 이동하여 정렬하므로 In-place 정렬에 해당한다. 힙 정렬 역시 입력 배열 자체를 힙으로 구성하여 정렬하므로 In-place 정렬에 해당한다. 반면 병합 정렬은 병합 과정에서 추가적인 임시 배열을 사용하므로 일반적인 배열 기반 구현에서는 In-place 정렬이 아니다.

알고리즘	In-place 여부
삽입 정렬	○
병합 정렬	×
힙 정렬	○

3.5 입력 데이터 특성에 따른 성능 차이

정렬 알고리즘은 입력 데이터의 정렬 상태에 따라서도 성능 차이가 발생할 수 있다.

삽입 정렬은 이러한 영향을 크게 받는 알고리즘으로, 이미 정렬되어 있거나 거의 정렬된 데이터에서는 원소의 이동이 적어 효율적으로 동작한다. 반면 내림차순과 같이 정렬되지 않은 데이터에서는 많은 원소를 이동해야 하므로 성능이 저하될 수 있다. 따라서 삽입 정렬은 최선의 경우 $O(n)$, 평균 및 최악의 경우 $O(n^2)$ 의 시간 복잡도를 가진다.

병합 정렬과 힙 정렬은 입력 데이터의 초기 정렬 상태와 관계없이 일반적으로 $O(n \log$

n)의 시간 복잡도를 유지한다.

입력 데이터	삽입 정렬	병합 정렬	힙 정렬
정렬된 데이터	$O(n)$	$O(n \log n)$	$O(n \log n)$
랜덤 데이터	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
역순 데이터	$O(n^2)$	$O(n \log n)$	$O(n \log n)$

따라서 입력 데이터의 상태에 따라 성능이 크게 달라질 수 있는 삽입 정렬과 달리, 병합 정렬과 힙 정렬은 데이터의 초기 상태에 상대적으로 영향을 적게 받는다는 차이가 있다.

4. 결론

본 보고서에서는 삽입 정렬, 병합 정렬, 힙 정렬을 대상으로 시간 복잡도, 공간 복잡도, 안정성, In-place 여부 및 입력 데이터의 특성에 따른 성능을 비교하였다.

실험 결과, 데이터의 크기가 증가할수록 세 알고리즘의 실행시간이 모두 증가하였으나, 특히 삽입 정렬에서 실행시간이 크게 증가하는 것을 확인하였다. 10,000개의 데이터를 대상으로 한 실험에서 삽입 정렬은 평균 **2788.813471 ms**, 병합 정렬은 **34.484261 ms**, 힙 정렬은 **35.411480 ms**가 소요되었다. 이를 통해 이론적으로 $O(n^2)$ 의 시간 복잡도를 가지는 삽입 정렬과 $O(n \log n)$ 의 시간 복잡도를 가지는 병합 정렬 및 힙 정렬의 차이가 실제 실행시간에서도 나타나는 것을 확인하였다.

공간 복잡도에서는 삽입 정렬과 힙 정렬이 추가적인 큰 메모리를 사용하지 않아 **$O(1)$** 의 추가 공간을 가지는 반면, 병합 정렬은 병합 과정에서 임시 배열을 사용하여 **$O(n)$** 의 추가 공간이 필요하였다. 또한 안정성 실험에서는 삽입 정렬과 병합 정렬이 동일한 값을 가진 원소들의 기존 순서를 유지하여 Stable Sort로 확인되었으며, 힙 정렬은 동일한 원소들의 순서가 변경되어 안정성이 보장되지 않음을 확인하였다.

또한 입력 데이터의 상태에 따라 삽입 정렬의 성능이 달라질 수 있음을 확인하였다. 삽입 정렬은 이미 정렬되어 있거나 거의 정렬된 데이터에서는 효율적으로 동작하지만, 역순 데이터에서는 많은 원소의 이동이 발생한다. 반면 병합 정렬과 힙 정렬은 입력 데이터의 상태와 관계없이 비교적 일정한 시간 복잡도를 유지한다.

종합적으로 세 알고리즘은 각각 다른 특징을 가지고 있으며, **데이터의 크기, 입력 상태, 추가 메모리 사용 여부 및 안정성 등을 고려하여 적절한 알고리즘을 선택할 필요가 있다.** 이번 비교를 통해 각 정렬 알고리즘의 이론적인 특성이 실제 실행 결과와 어떻게 연결되는지 확인할 수 있었다.

깃허브 링크: minjik070316-creator/work_due0930